Technical Report

Submission-to-Decision Traceability Platform

Sydney Erik Wamalwa · Capstone project · Evidence snapshot: 29 October 2026

1 Scope Alignment and Feature Validation

The approved proposal's core workflow is implemented: citizens submit views against tracked legislation; the platform issues receipts; committee reports are ingested; candidate submission-to-report links are proposed; a human reviewer confirms or rejects a match; and confirmed treatment is made available through receipt and bill outcome views. Representative adoption is optional—the accountability model relies on institution-published committee reports. Statuses below distinguish code implementation from deployment or evaluation evidence.

Planned feature	Purpose	Status	Evidence / caveat
Parliament bill tracking	Discover bills, stages and source documents.	Implemented	`backend/app/scripts/sync_legislation.py`; `backend/tests/test_sync_legislation.py`. Live API snapshot returned 676 bills, all `is_simulated=false`.
Submission, receipt, integrity	Record a citizen view and provide a verifiable timestamped receipt.	Implemented; hosted integrity path not fully tested	`src/pages/SubmitViewPage.tsx`; `backend/app/services/receipt_service.py`; Alembic integrity migration.
Report ingestion, OCR, matching, review	Extract committee treatments and propose auditable links; reserve final decision for a moderator.	Implemented; accuracy evaluation pending	`backend/app/services/ingestion_service.py`, `linking_service.py`; `src/pages/ModeratorPages.tsx`; synthetic PDF fixtures only.
Outcome, notices, profiles	Show receipt status/outcome trail, participation notice details, and optional office profiles.	Implemented for pilot	`src/pages/ReceiptPages.tsx`, `BillsPages.tsx`, `NoticesPage.tsx`, `ProfilesPages.tsx`; real report coverage depends on sources.
Offline drafts (FR10)	Enable drafting during intermittent connectivity and later synchronization.	Partial verification	`src/utils/offline.ts`; auto-sync is not yet verified against a live API.
Lodging and evaluation	Support institutional lodging evidence; assess usability and extraction/link quality.	Manual / evaluation pending	`docs/DECISIONS.md` notes lodging-channel integration is manual; evaluation scripts/templates exist, but approved accuracy results and the human SUS study are not complete.

The currently hosted bill catalogue is drawn from the Parliament legislation sync, not the seeded demo-bill dataset. Seed/demo records remain available for local testing and must be distinguished from Parliament-sourced bills and from actual institutional committee outcomes. The proposal's broader national rollout, mass representative onboarding, polls/forums, leader scoring and binding online voting are outside the bounded pilot scope. A platform receipt is evidence of platform recording, not proof of official Assembly receipt.

2 Technical Functionality and Architecture

The hosted design is a React/TypeScript and Vite frontend on Netlify, a FastAPI/Python REST API on Render, and PostgreSQL on Neon. The browser calls the API; server-side routes enforce role permissions and delegate domain work to services; the persistence layer stores bills, receipts, report data, notices and review decisions. Frontend API client modules are under `src/api/`; backend models/routes are in `backend/app/models.py` and `backend/app/api/routes.py`.

Receipt creation and chain verification use SHA-256-linked records; an Alembic migration adds database-level immutability controls. PDF ingestion extracts native text and uses an OCR fallback for scanned reports. The linking service prioritizes clause references and uses text similarity as a fallback; an optional model proposal is not authoritative. Low-confidence cases can abstain, and a human moderator confirms or rejects proposed matches before publication. Other implemented pilot capabilities include role-based workflows, notice records, profiles, moderation/reporting, anonymized export and local offline drafts. Automated lodging through an Assembly portal or email is not implemented; that step remains manual.

3 Testing Approaches and Results

1. Automated checks (type/build and backend test suite). The frontend checks were run in the extracted repository with the locked npm dependencies. `npm run typecheck` passed. `npm run build` passed and produced a Vite production bundle. The build emitted a chunk-size warning: the main minified JavaScript chunk was about 1,295 kB (about 369 kB gzip), above Vite's 500 kB advisory threshold. The dependency install reported 5 audit vulnerabilities (3 moderate, 2 high); these require review with `npm audit` before release.

Backend tests were run from the repository root with a temporary test-only JWT secret and no `TEST_DATABASE_URL`: `PYTHONPATH=backend pytest -c backend/pytest.ini backend/tests/ -q`. Result: **20 passed, 19 skipped**. The passed tests include parser, authentication-hardening and legislation-sync checks. Database-dependent API/integration tests were skipped because no PostgreSQL test database was configured; skipped tests are not passing evidence. The OCR regression fixtures are synthetic. No approved-dataset precision/recall/F1 or human usability/SUS result is claimed.

2. Hosted HTTP smoke testing. On 29 October 2026, one non-invasive request was made to each hosted endpoint. Netlify homepage returned HTTP 200 in 3.57 s; Render `/health` returned HTTP 200 (`healthy`, version 1.0.0) in 3.97 s; Render `/api/v1/bills` returned HTTP 200 in 8.24 s with 676 bill records. All 676 were marked non-simulated; 674 included a source URL. An inspected record referenced a Parliament tracker PDF and carried source-update date 2026-10-29. This supports the current bill-source correction. These are single-request smoke timings, not a performance benchmark; they do not verify login, submission, receipt-chain verification, moderation persistence or Neon failover.

The next validation steps are to run database-dependent tests against an isolated PostgreSQL test database; complete an authenticated hosted journey (login → submit → receipt lookup/verification → moderator decision); measure repeat latency and compatibility on named browsers/devices; and complete ethics-approved usability and manually labelled ingestion/linking evaluations.

4 Code Quality Practices

The codebase separates concerns: typed React pages and reusable components handle presentation; `src/api/` isolates HTTP calls; FastAPI routes coordinate requests; SQLAlchemy models describe persisted entities; and services isolate receipt, ingestion, linking, notification and storage logic. This modularity makes domain behavior testable and avoids concentrating all functionality in a single script. TypeScript provides compile-time interface checks, while backend tests cover role restrictions, authentication hardening, parsing and sync behavior.

Technical problem-solving is tailored to the domain: deterministic SHA-256 receipt chaining; native PDF extraction with OCR fallback; clause-first/text-similarity candidate matching with confidence thresholds and human review; role-based authorization; and legislation sync that parses official tracker documents and matches bills using normalized identifiers. The matching score is an aid, not proof. Current quality risks evidenced by validation are the oversized frontend bundle, reported dependency advisories, and incomplete PostgreSQL-backed and human evaluation. These should be addressed before production claims.

Evidence basis: repository source and tests in the submitted archive; hosted Netlify/Render HTTP checks on 2026-10-29. Bill counts and timings are point-in-time observations and may change after a subsequent synchronization.